

McGill University
ECSE 222 – Digital Logic (Fall 2025)
Pen-and-Paper Assignment #6

Instructions

- TA in charge: Sihui Shen (sihui.shen@mail.mcgill.ca)
- Write/type your answers in the space provided.
- Due date: Monday, Dec 8th, 2025, by 11:59 p.m. EDT.
- You must electronically submit the assignment in the form of a PDF file (only this format will be accepted) on myCourses before the deadline. You may submit as many times as you want, however, only the last submission will be kept by the system.
- You must solve the assignment in teams of two students, whereas only one of the students should perform the actual electronic submission.
- Late submissions will incur penalties as described in the course syllabus.

Student identification

First student full name:	Aneurin Hullah
First student ID number:	261283866
Second student full name:	Alexandre Pujal
Second student ID number:	261183554
Third student full name (if applicable)	
Third student ID number (if applicable)	

Evaluation (for course graders use only)

Question 1 (20%)	Question 2 (10%)	Question 3 (20%)	Question 4 (20%)	Question 5 (15%)	Question 6 (15%)	Total

Perand Paper Assignment #6:

Name, Alexandre PUSAL

no: 261183554

Question 1:

CE	W/P	
0	0	1
0	1	0
1	0	0
1	1	1

select the mode of operation

Moreover, $\begin{cases} I/FF=0 \Rightarrow \text{Latch (denote the output } Q_0) \\ I/FF=1 \Rightarrow \text{FF (denote the output } Q_1) \end{cases}$

~~Q = Q_0~~

~~Q = Q_0 or Q_1 and of CE, Q = Q_0 if CE=0, Q = Q_1 if CE=1~~

~~$I/FF=0 \Rightarrow Q = Q_0$~~
 ~~$I/FF=1 \Rightarrow Q = Q_1$~~

	I/FF	Q ₀	Q ₁	Q
0	0	0	0	0
1	0	0	1	0
2	0	1	0	1
3	0	1	1	1
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
7	1	1	1	1

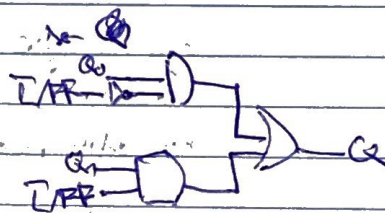
From the b-may,

$$Q = (I/FF Q_0) + (I/FF Q_1)$$

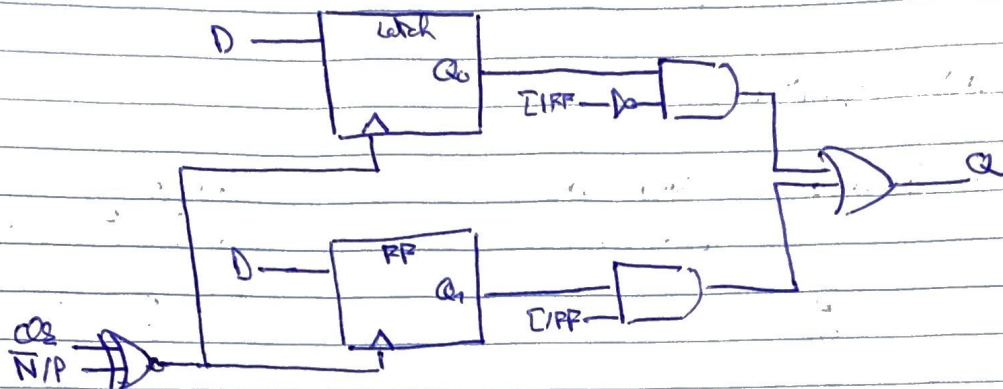
I/FF Q₀

Q ₀	Q ₁	Q
0	0	0
1	0	0
0	1	0
1	1	1

Q ₀	Q ₁	Q
0	0	0
1	0	0
0	1	0
1	1	1

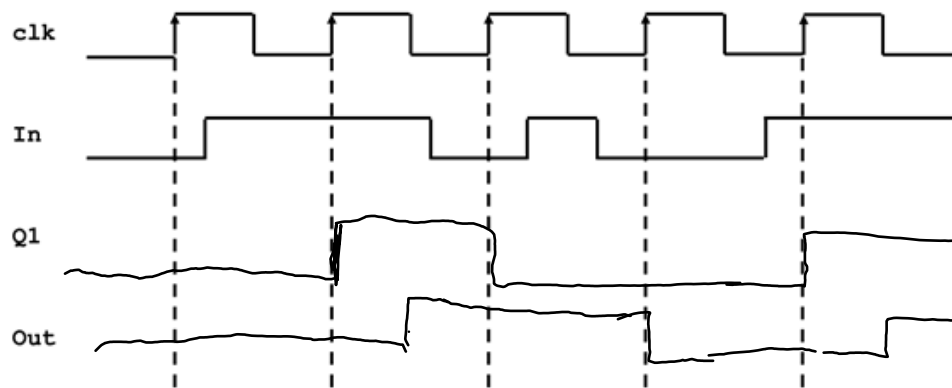
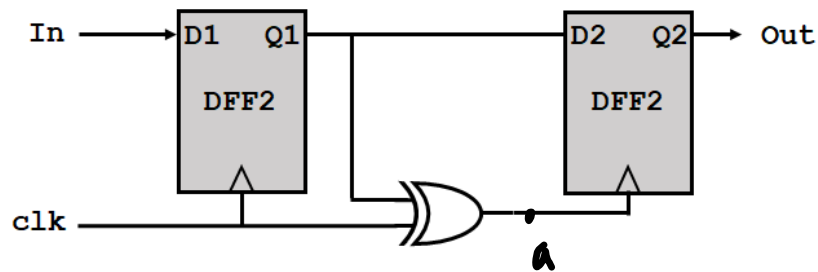


Here, the final implementation is the following:



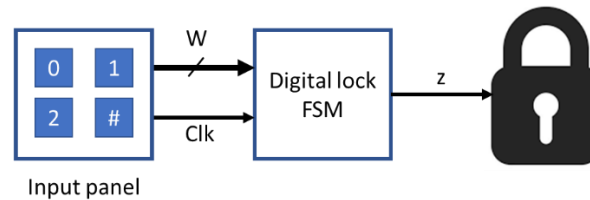
Question 2 (10%)

For the following circuit, complete the timing diagram below. Provide your answer on the given timing diagram. Assume ideal components (no latency). Initial state $Q1Q2 = "00"$.



Question 3 (20%)

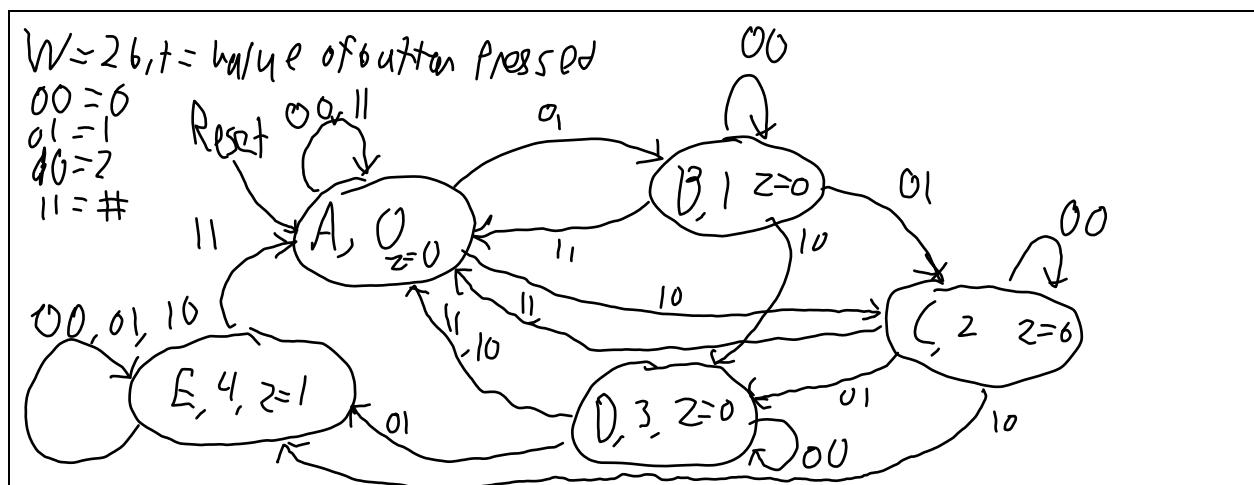
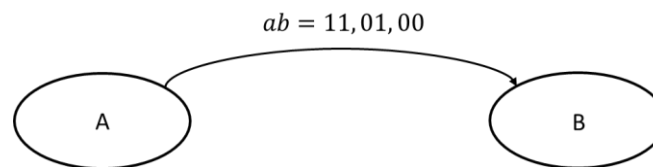
Design a digital lock system. The system includes three components (as depicted in the figure below): (1) an input panel that consists of four buttons, (2) an FSM that receives the punched-in code 'W' in binary form, and generates an output signal 'z', and (3) a lock that unlocks when $z = 1$.



Specifications:

- When any button is pressed on the input panel, a binary sequence 'W' is generated and sent to the FSM. At the same time, a single clock pulse (rise and fall) is also generated and sent to the FSM, assume that all timing constraints are met.
- The digital lock includes a summation unit such that the lock only opens when the sum of all pressed numbers is 4, producing an output $z = 1$, otherwise $z = 0$. If the value in the summation unit exceeds 4, it automatically resets to zero.
- The '#' button is used to lock the system and reset the summation unit in the FSM to zero.
- Assume that the initial state of the system is in the locked condition and the value in the summation unit is zero.
- Assume that once unlocked, the system stays unlocked until the '#' button is pressed.

Design and implement the digital lock FSM as a Moore machine using DFFs. You must determine the size of 'W' and provide all the steps of your solution. Note that if two different input combinations point from one state to the same next state, they may be combined on a single edge as illustrated here:



State table

Present State	W	00	01	10	11	Z
A	A	B	C	A	0	
B	B	C	D	A	0	
C	C	D	E	A	0	
D	D	E	A	A	0	
E	E	E	E	A	1	

State assigned table

Y_2, Y_1, Y_0	00	01	10	11	Z
000	000	001	010	000	0
001	001	010	011	000	0
010	010	011	100	000	0
011	011	100	000	000	0
100	100	100	100	000	1

Next State Karnaugh Maps

Y_0 K-map:

Y_1, Y_0	00	01	11	10
00	0	1	1	0
01	1	0	0	1
11	0	0	0	0
10	0	1	0	0

Y_1 K-map:

Y_1, Y_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	0	0
10	0	0	0	0

Output Equations

$Z = Y_2$

$Y_0 = Y_1 \bar{W}_1 \bar{W}_0 + Y_1 \bar{Y}_0 \bar{W}_1 W_0 + \bar{Y}_1 Y_0 W_1 \bar{W}_0 + \bar{Y}_1 Y_0 \bar{W}_1 W_0 \bar{Y}_2$

Y_1 K-map:

Y_1, Y_0	00	01	11	10
00	0	0	1	1
01	0	1	0	1
11	0	0	0	0
10	1	1	0	0

$Y_1 = Y_1 \bar{Y}_0 \bar{W}_1 + Y_1 \bar{W}_1 \bar{W}_0 \bar{Y}_2 + \bar{Y}_1 Y_0 \bar{W}_1 W_0 + \bar{Y}_1 W_1 \bar{W}_0 \bar{Y}_2$

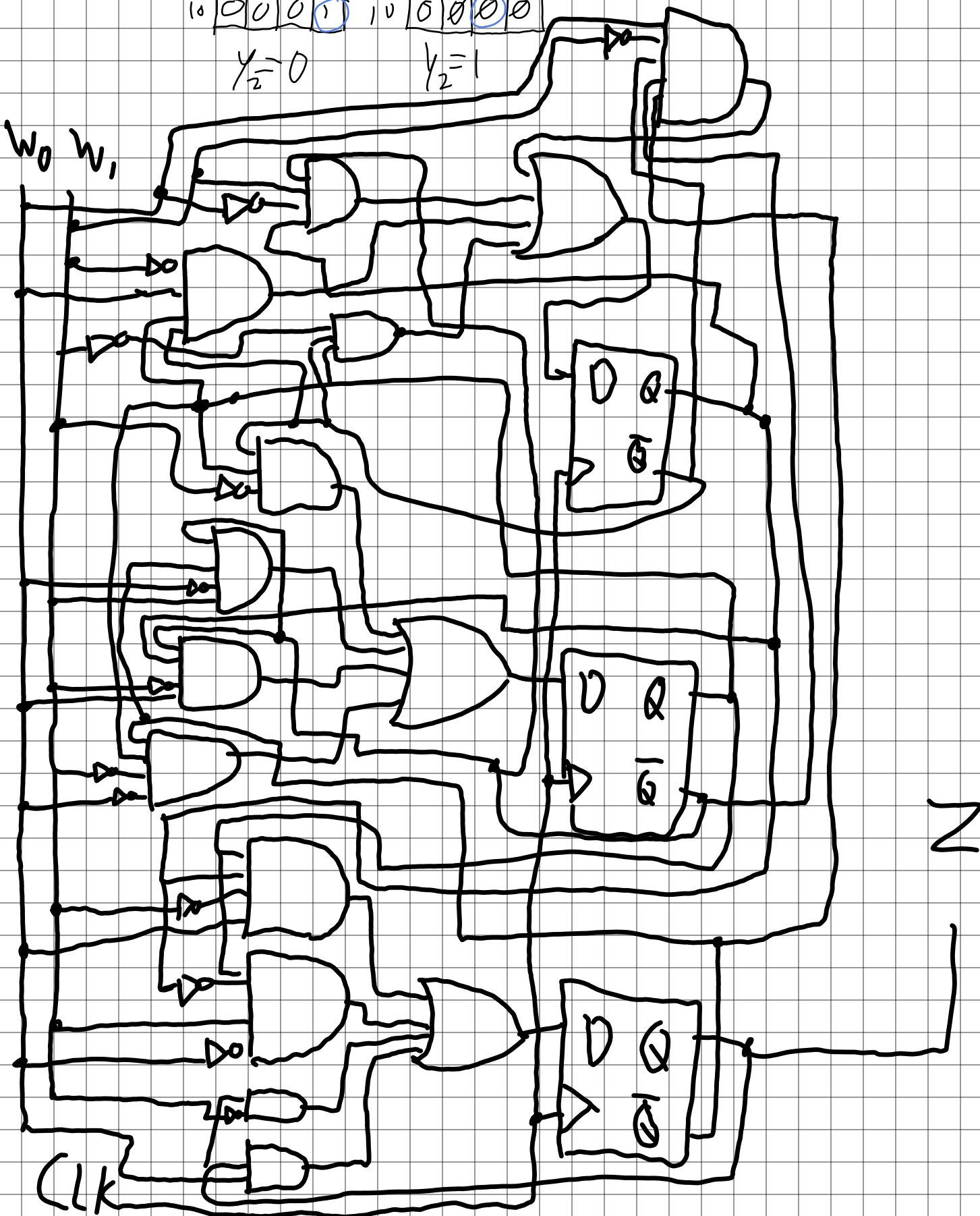
Y
2

		y_1, y_0						y_1, y_0							
		w_1, w_0		00	01	11	10			w_1, w_0		00	01	11	10
w_1, w_0	00	0	0	0	0	0	0	00	1	0	0	0	0	0	0
	01	0	0	1	0	0	0	01	1	0	0	0	0	0	0
	11	0	0	0	0	0	0	11	1	0	0	0	0	0	0
	10	0	0	0	1	0	0	10	0	0	0	0	0	0	0

$y_2 = 0$

$y_2 = 1$

$$Y_2 = y_1 y_0 \bar{w}_1 w_0 + y_1 \bar{y}_0 w_1 \bar{w}_0 + y_2 \bar{w}_1 + y_2 w_0$$

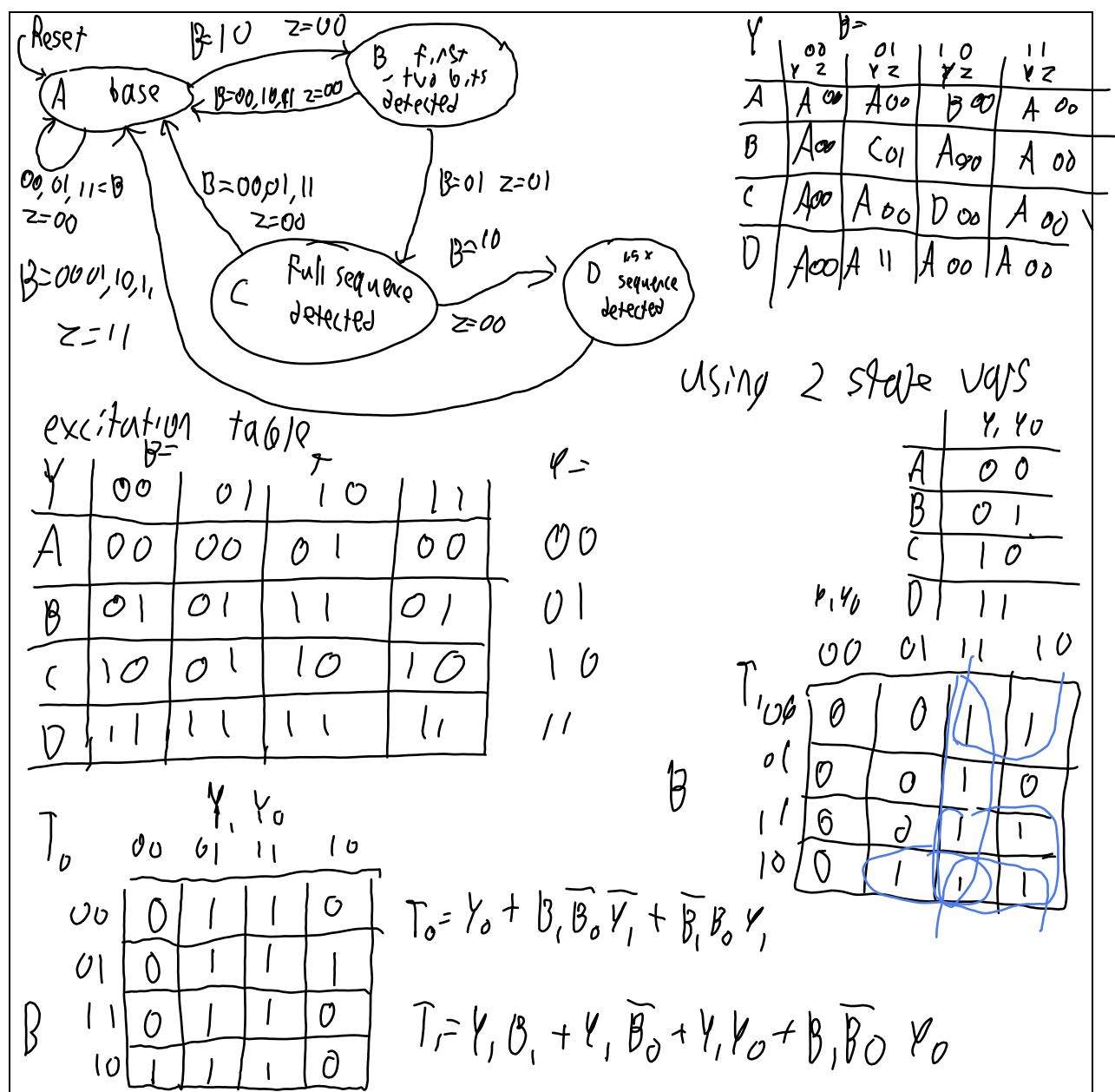


Question 4 (20%)

A non-overlapping sequence detector is a machine that resets after detecting the desired sequence. Generate a state diagram (clearly explain the meaning of each state), relevant tables, and circuit for a non-overlapping “1001” sequence detector with the following specifications:

- Machine has a two-bit input $\{b_1b_0\}$: every clock cycle two bits of the sequence arrive
- MSB of the sequence should be detected first
- This machine has two outputs z_1z_0
- Machine will output $z_0 = 1$ as soon as the sequence is detected (in the same clock cycle)
- Machine will output $z_1 = 1$ as soon as the sequence is detected twice in a row (in the same clock cycle)

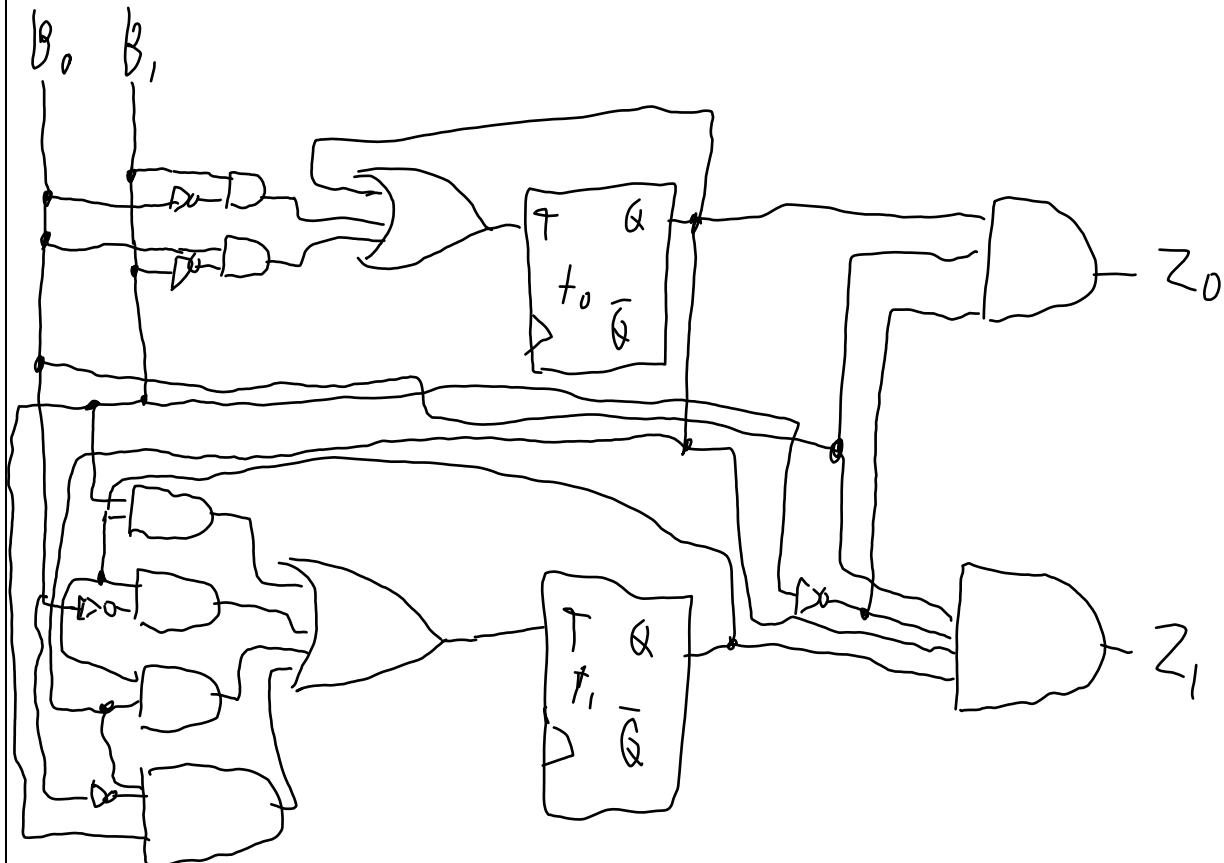
Use as few Toggle FFs (TFFs) as possible and logic gates as needed.



$$Z_0 = \overline{B_1} B_0 \overline{Y_1} Y_0 + \overline{B_1} B_0 Y_1 Y_0 = \overline{B_1} B_0 Y_0$$

$$Z_1 = \overline{B_1} B_0 Y_1 Y_0$$

circuit w/ 2 TFFs



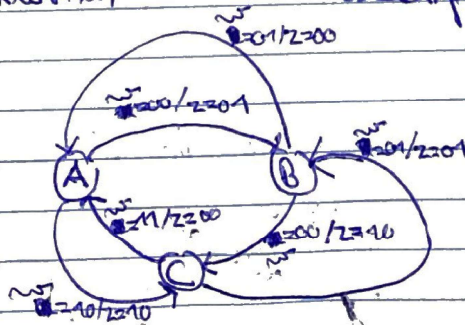
Question 5:

input: x_1, x_2
output: z_1, z_2

input action	x_1	x_2
one employee goes in	0	0
one employee goes out	0	1
two employees go in	1	0
two employees go out	1	1

States:
A: no one in
B: one in
C: two in

output z	z_1	z_2
no one in	0	0
one in	0	1
two in	1	0
$\emptyset$	1	1



State Table:

Present state	Next state				Output z			
	$w_3w_2w_1w_0$	$w_3w_2w_1w_0$	$w_3w_2w_1w_0$	$w_3w_2w_1w_0$	$w_3w_2w_1w_0$	$w_3w_2w_1w_0$	$w_3w_2w_1w_0$	$w_3w_2w_1w_0$
A	B	$\emptyset$	C	$\emptyset$	01	$\emptyset$	10	$\emptyset$
B	C	A	$\emptyset$	$\emptyset$	10	00	$\emptyset$	$\emptyset$
C	$\emptyset$	B	$\emptyset$	A	$\emptyset$	01	$\emptyset$	∞

State-Assigned Table:

Present state	Next state				Output z			
	y_1y_0	x_1x_0	y_1y_0	x_1x_0	y_1y_0	x_1x_0	y_1y_0	x_1x_0
A	00	01	01	10	01	$\emptyset$	10	$\emptyset$
B	01	10	00	$\emptyset$	10	00	$\emptyset$	$\emptyset$
C	10	$\emptyset$	01	$\emptyset$	$\emptyset$	01	$\emptyset$	∞

Next-state logic:

Output logic:

y_1y_0	$w_3w_2w_1w_0$	x_1x_0
00	0000	01
00	0001	01
00	0010	01
00	0011	01
00	0100	01
00	0101	01
00	0110	01
00	0111	01
00	1000	01
00	1001	01
00	1010	01
00	1011	01
00	1100	01
00	1101	01
00	1110	01
00	1111	01

y_1y_0	$w_3w_2w_1w_0$	z
00	0000	0
00	0001	0
00	0010	0
00	0011	0
00	0100	0
00	0101	0
00	0110	0
00	0111	0
00	1000	0
00	1001	0
00	1010	0
00	1011	0
00	1100	0
00	1101	0
00	1110	0
00	1111	0

$$x_1 = y_1w_0 + y_0w_1$$

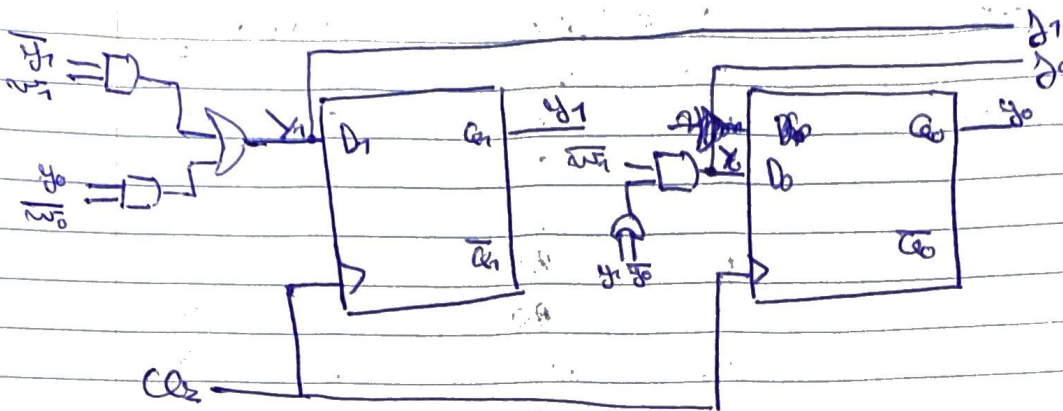
$$z = y_1w_1 + y_0w_0 = w_1(y_1 + y_0)$$

maps:

y_1y_0	00	01	11	10
$w_3w_2w_1w_0$	00 0	01 1	11 0	10 0
01	0	0	0	0
11	0	0	0	0
10	1	0	0	0

y_1y_0	00	01	11	10
$w_3w_2w_1w_0$	00 1	01 0	11 0	10 0
01	0	0	0	0
11	0	0	0	0
10	0	0	0	0

Hence the final implementation would be:



Question 6:

~~State transition diagram~~
~~A: only one bit (A:1, B:0)~~
~~B: only two bits (A:0, B:1)~~
~~C: only three bits (A:1, B:1, C:0)~~
~~D: only four bits (A:1, B:1, C:1, D:0)~~
~~E: only five bits (A:1, B:1, C:1, D:1, E:0)~~
~~F: only six bits (A:1, B:1, C:1, D:1, E:1, F:0)~~
~~G: only seven bits (A:1, B:1, C:1, D:1, E:1, F:1, G:0)~~
~~H: only eight bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:0)~~
~~I: only nine bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:0)~~
~~J: only ten bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:0)~~
~~K: only eleven bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:0)~~
~~L: only twelve bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:0)~~
~~M: only thirteen bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:1, M:0)~~
~~N: only fourteen bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:1, M:1, N:0)~~
~~O: only fifteen bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:1, M:1, N:1, O:0)~~
~~P: only sixteen bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:1, M:1, N:1, O:1, P:0)~~
~~Q: only seventeen bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:1, M:1, N:1, O:1, P:1, Q:0)~~
~~R: only eighteen bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:1, M:1, N:1, O:1, P:1, Q:1, R:0)~~
~~S: only nineteen bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:1, M:1, N:1, O:1, P:1, Q:1, R:1, S:0)~~
~~T: only twenty bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:1, M:1, N:1, O:1, P:1, Q:1, R:1, S:1, T:0)~~
~~U: only twenty-one bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:1, M:1, N:1, O:1, P:1, Q:1, R:1, S:1, T:1, U:0)~~
~~V: only twenty-two bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:1, M:1, N:1, O:1, P:1, Q:1, R:1, S:1, T:1, U:1, V:0)~~
~~W: only twenty-three bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:1, M:1, N:1, O:1, P:1, Q:1, R:1, S:1, T:1, U:1, V:1, W:0)~~
~~X: only twenty-four bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:1, M:1, N:1, O:1, P:1, Q:1, R:1, S:1, T:1, U:1, V:1, W:1, X:0)~~
~~Y: only twenty-five bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:1, M:1, N:1, O:1, P:1, Q:1, R:1, S:1, T:1, U:1, V:1, W:1, X:1, Y:0)~~
~~Z: only twenty-six bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:1, M:1, N:1, O:1, P:1, Q:1, R:1, S:1, T:1, U:1, V:1, W:1, X:1, Y:1, Z:0)~~

States:

A, B: only one bit (A:1, B:0)

C, D: only two bits (A:0, B:1)

E, F: only three bits (A:1, B:1, C:0)

G, H: only four bits (A:1, B:1, C:1, D:0)

I, J: only five bits (A:1, B:1, C:1, D:1, E:0)

K, L: only six bits (A:1, B:1, C:1, D:1, E:1, F:0)

M, N: only seven bits (A:1, B:1, C:1, D:1, E:1, F:1, G:0)

O, P: only eight bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:0)

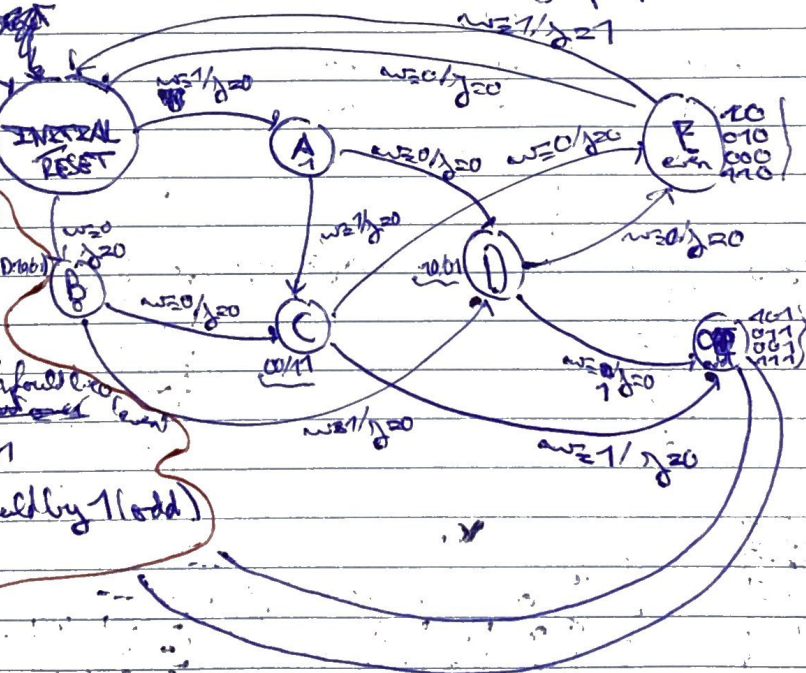
Q, R: only nine bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:0)

S, T: only ten bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:0)

U, V: only eleven bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:0)

W, X: only twelve bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:0)

Y, Z: only thirteen bits (A:1, B:1, C:1, D:1, E:1, F:1, G:1, H:1, I:1, J:1, K:1, L:1, M:0)



z:

		wyz			
		00	01	11	10
00	0	0	0	0	0
01	0	1	0	0	0
11	0	0	1	0	0
10	0	0	0	1	0

$$z = w'y_2y_0$$

$$z = wy_1y_0' + wy_2y_0 + y_2'y_0'$$

$$z = w'y_1y_0' + wy_2y_0 + wy_2y_1y_0$$

$$z = y_2y_1y_0' + w'y_2y_0 + y_2y_0 + wy_2y_1$$

